

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Разработка веб-клиента для программной АТС на базе FreeSWITCH

Курсовая работа

09.03.04 Программная инженерия

Информационные системы и сетевые технологии

Зав. кафедрой _____ С.Д. Махортов, д.ф.-м.н., доцент _____.20__

Обучающийся _____ М.Е. Немченко, 3 курс, д/о

Руководитель _____ А.И. Чекмарев, ст. преподаватель

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	3
ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	7
1.1. Понятие программной АТС и её роль в современных компаниях	7
1.2. Обзор FreeSWITCH как серверной платформы.....	8
1.3. Анализ существующих веб-клиентов для АТС	9
1.4. Обоснование необходимости разработки нового веб-клиента	11
1.5. Постановка задачи и требования к системе	12
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ	15
2.1. Архитектура приложения (Feature-Sliced Design)	15
2.2. Выбор и обоснование стека технологий	16
2.3. Проектирование компонентной структуры	19
2.4. Проектирование взаимодействия с сервером	21
2.5. Модель данных и описание основных сущностей	23
ГЛАВА 3. РЕАЛИЗАЦИЯ ВЕБ-КЛИЕНТА	26
3.1. Структура проекта и слои FSD	26
3.2. Реализация авторизации и управления сессией.....	27
3.3. Реализация отображения операторов и их статусов.....	28
3.4. Реализация обработки входящих и исходящих звонков	31
3.5. Интеграция WebRTC через протокол Verto	34
3.6. Система хранения данных	35
ГЛАВА 4. ТЕСТИРОВАНИЕ И РЕЗУЛЬТАТЫ	38
4.1. Описание тестового стенда.....	38
4.2. Функциональное тестирование.....	38
ЗАКЛЮЧЕНИЕ	40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
ПРИЛОЖЕНИЕ А.....	44

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей курсовой работе применяются следующие термины, обозначения и сокращения.

Колл-центр — подразделение компании, осуществляющее приём и обработку входящих и исходящих телефонных вызовов.

STUN (Session Traversal Utilities for NAT) — протокол, обеспечивающий прохождение медиапотокa WebRTC через сетевые экраны и NAT.

CDR (Call Detail Record) — запись с детальной информацией о завершённом вызове: время, длительность, участники, тип вызова.

FSD (Feature-Sliced Design) — методология архитектурной организации фронтенд-проекта, предполагающая разделение кода на слои по бизнес-доменам.

Redux — библиотека для централизованного управления состоянием клиентского приложения.

TypeScript — язык программирования, расширяющий JavaScript статической типизацией.

React — JavaScript-библиотека для построения пользовательских интерфейсов на основе компонентного подхода.

HTTP REST API — интерфейс взаимодействия с сервером по протоколу HTTP, используемый для авторизации и загрузки данных.

Verto — JSON-RPC-протокол, разработанный командой FreeSWITCH, работающий поверх WebSocket и обеспечивающий управление вызовами через браузер.

WebSocket — протокол полнодуплексного обмена сообщениями между браузером и сервером по постоянному соединению.

WebRTC (Web Real-Time Communications) — технология, позволяющая организовывать голосовую и видеосвязь непосредственно в браузере без установки дополнительного программного обеспечения.

SIP (Session Initiation Protocol) — протокол сигнализации для установления, изменения и завершения мультимедийных сессий в IP-сетях.

FreeSWITCH — свободная масштабируемая платформа телефонии с открытым исходным кодом, используемая в качестве серверной части системы.

Веб-клиент — браузерное приложение, предоставляющее оператору интерфейс для работы с программной АТС.

АТС — автоматическая телефонная станция — система коммутации, обеспечивающая установление, поддержание и завершение телефонных соединений между абонентами.

ВВЕДЕНИЕ

Одним из ключевых элементов корпоративной инфраструктуры являются автоматические телефонные станции (АТС) — системы, обеспечивающие маршрутизацию, обработку и контроль телефонных вызовов. В последние годы традиционные аппаратные АТС всё активнее вытесняются программными решениями.

Особую роль среди платформ для построения телефонных систем занимает FreeSWITCH — высокопроизводительная масштабируемая платформа с открытым исходным кодом, предназначенная для обработки голосовых и мультимедийных коммуникаций. FreeSWITCH получила широкое распространение благодаря своей гибкости, поддержке современных протоколов (SIP, WebRTC, Verto) и возможности интеграции с различными клиентскими приложениями.

Неотъемлемой частью подобных систем является пользовательский интерфейс — веб-клиент, через который операторы колл-центра выполняют свою повседневную работу: принимают входящие вызовы, совершают исходящие, переводят звонки на других операторов и управляют своим рабочим статусом. От качества веб-клиента во многом зависит производительность труда оператора и удобство эксплуатации всей системы в целом.

В рамках данной курсовой работы рассматривается задача разработки современного веб-клиента для программной АТС на базе FreeSWITCH. Существующий веб-клиент был реализован с использованием библиотеки jQuery — подхода, который, несмотря на свою историческую популярность, перестал соответствовать требованиям современной веб-разработки. Управление DOM-деревом вручную, отсутствие компонентного подхода, сложность поддержки кода при масштабировании — всё это обусловило необходимость перехода на более современный технологический стек.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) изучить устройство программной АТС и принципы взаимодействия с ней через протоколы HTTP, WebSocket и WebRTC (Verto);
- 2) провести анализ существующих решений;
- 3) спроектировать архитектуру клиентского приложения;
- 4) реализовать модули авторизации, истории вызовов, отображения операторов и управления их статусами;
- 5) реализовать функциональность обработки входящих и исходящих звонков с передачей аудио через WebRTC;
- 6) обеспечить обновление данных в реальном времени посредством WebSocket-соединения;
- 7) провести функциональное тестирование разработанного приложения и оценить соответствие результатов поставленным требованиям.

Объектом исследования является веб-клиент для программной АТС на базе FreeSWITCH. Предметом исследования выступают технологии разработки браузерного интерфейса для взаимодействия в режиме реального времени.

Практическая значимость работы заключается в том, что разработанный веб-клиент представляет собой готовое к эксплуатации приложение, а также служит основой для дальнейшего развития и расширения функциональности системы. Применённые архитектурные решения и технологии обеспечивают высокую масштабируемость и удобство сопровождения кода.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Понятие программной АТС и её роль в современных компаниях

Автоматическая телефонная станция представляет собой систему, обеспечивающую установление, поддержание и завершение телефонных соединений между абонентами. Исторически АТС реализовывались в виде аппаратных комплексов, требующих значительных капитальных вложений, специализированного технического персонала и физической инфраструктуры. Однако с развитием технологий передачи голоса по IP-сетям (VoIP) и широким распространением мощных серверных платформ произошёл постепенный переход от аппаратных к программным решениям.

Программная АТС — это программный комплекс, реализующий функции традиционной телефонной станции на серверном оборудовании. В отличие от аппаратных аналогов, программные АТС не привязаны к специализированному «железу» и могут быть развёрнуты как на физическом сервере, так и в облачной инфраструктуре. Гибкость конфигурирования и значительно меньшая стоимость владения сделали программные АТС основным инструментом построения корпоративной телефонии.

Особую роль программные АТС играют в организации работы колл-центров и контакт-центров. Именно здесь максимально востребованы такие возможности, как равномерное распределение входящего потока звонков между операторами, мониторинг загрузки персонала в режиме реального времени, оперативное переключение вызовов между сотрудниками и формирование отчётности для управленческих решений. Эффективность работы колл-центра во многом определяется качеством программного обеспечения, используемого операторами — прежде всего, удобством и функциональностью веб-клиента АТС.

Современные программные АТС, как правило, реализуют поддержку протокола SIP (Session Initiation Protocol) — стандарта де-факто для IP-телефонии, обеспечивающего сигнализацию и управление мультимедийными

сессиями. Помимо SIP, перспективным направлением является поддержка WebRTC (Web Real-Time Communications) — технологии, позволяющей организовывать голосовую и видеосвязь непосредственно в браузере без установки дополнительного программного обеспечения на рабочей станции оператора.

Таким образом, программная АТС является центральным элементом современной корпоративной телефонной инфраструктуры, обеспечивающим не только коммутацию вызовов, но и комплексное управление коммуникационными процессами предприятия.

1.2. Обзор FreeSWITCH как серверной платформы

FreeSWITCH — это свободная масштабируемая платформа телефонии с открытым исходным кодом, разработанная Энтони Минессейлом и выпущенная в 2006 году. Платформа написана преимущественно на языке С и распространяется под лицензией Mozilla Public License (MPL). Проект активно поддерживается сообществом разработчиков, а также компанией SignalWire — коммерческим спин-оффом, основанным создателями FreeSWITCH.

Архитектура FreeSWITCH построена по модульному принципу: ядро платформы обеспечивает базовую обработку вызовов, тогда как расширенная функциональность реализована в виде подключаемых модулей. Модули FreeSWITCH охватывают широкий спектр функций: поддержку различных протоколов сигнализации (SIP, H.323, Skinny), кодеков, форматов записи, хранилищ данных и интерфейсов управления.

Одним из ключевых конкурентных преимуществ FreeSWITCH является поддержка протокола WebRTC и разработанного командой FreeSWITCH протокола Verto (VERsaTile Object). Verto — это JSON-RPC-протокол, работающий поверх WebSocket, специально спроектированный для организации взаимодействия между браузерным клиентом и FreeSWITCH. Он обеспечивает передачу управляющих сообщений (установление, завершение, перевод вызова), а медиапоток передаётся напрямую через WebRTC. Это

делает FreeSWITCH идеальной платформой для построения веб-ориентированных решений для телефонии.

FreeSWITCH предоставляет несколько интерфейсов управления, используемых при разработке клиентских приложений. Event Socket Library (ESL) — это TCP-интерфейс, позволяющий внешним приложениям подписываться на события FreeSWITCH (поступление вызова, изменение состояния канала, завершение соединения и другие), а также отправлять команды управления. HTTP REST API, реализуемый через модули `mod_xml_rpc` или `mod_httapi`, позволяет выполнять операции по стандартным HTTP-протоколам. Протокол Verto через WebSocket обеспечивает полноценное взаимодействие с браузерным клиентом.

С точки зрения производительности FreeSWITCH способен обслуживать тысячи одновременных вызовов на одном сервере, что делает его пригодным для развёртывания как в небольших организациях, так и в крупных телекоммуникационных проектах. Платформа поддерживает работу в режиме высокой доступности с кластеризацией и балансировкой нагрузки.

В контексте разрабатываемого веб-клиента FreeSWITCH выступает серверной частью системы: он обрабатывает вызовы, управляет состоянием операторов и предоставляет события через WebSocket-соединение. Веб-клиент взаимодействует с FreeSWITCH через три канала: HTTP REST API для операций авторизации и загрузки данных, WebSocket для получения событий в реальном времени и WebRTC (через Verto) для непосредственной передачи голосового трафика.

1.3. Анализ существующих веб-клиентов для АТС

Анализ существующих решений позволяет выявить ключевые функциональные требования к подобным системам, а также определить преимущества и недостатки различных подходов к их реализации.

Среди наиболее распространённых коммерческих решений следует отметить 3CX Web Client, Zoiper Web и Bitrix24 Телефония. Данные продукты

предоставляют полнофункциональный интерфейс оператора, включая управление вызовами, статусами, историей звонков и интеграцию с корпоративными системами. Однако их существенным недостатком является закрытость: они работают исключительно со своей АТС-платформой и не могут быть напрямую интегрированы с FreeSWITCH.

В экосистеме FreeSWITCH наиболее известным проектом является Verto Communicator — официальный веб-клиент, разработанный командой FreeSWITCH для демонстрации возможностей протокола Verto. Он реализован на базе AngularJS и предоставляет базовые функции совершения и приёма вызовов через WebRTC. Несмотря на свою историческую значимость, Verto Communicator имеет ряд существенных ограничений: устаревший технологический стек (AngularJS официально переведён в режим долгосрочной поддержки с прекращением активного развития), отсутствие развитого интерфейса управления операторами и ограниченные возможности кастомизации.

Проект FusionPBX — популярная система управления FreeSWITCH с открытым исходным кодом — включает встроенный веб-клиент на базе JavaScript без использования современных фреймворков. Он обеспечивает базовые функции телефонии, однако страдает от тех же проблем устаревших подходов: сложность сопровождения кода, ограниченная реактивность интерфейса и отсутствие поддержки современных стандартов разработки.

Для понимания контекста данной курсовой работы особую важность имеет анализ исходного веб-клиента, разработанного на базе jQuery. jQuery — это JavaScript-библиотека, упрощающая работу с DOM, AJAX-запросами и анимациями. С появлением современных реактивных фреймворков её применение для построения сложных интерактивных приложений стало нецелесообразным.

Сравнительный анализ существующих решений позволяет сформулировать ключевые характеристики, которым должен соответствовать современный веб-клиент для программной АТС: компонентная архитектура,

обеспечивающая масштабируемость и удобство сопровождения; реактивное обновление интерфейса без перезагрузки страницы; поддержка WebRTC для передачи голоса в браузере; централизованное управление состоянием приложения; типобезопасность кода; поддержка актуальных браузеров без дополнительных плагинов.

1.4. Обоснование необходимости разработки нового веб-клиента

Исходный веб-клиент системы был реализован с использованием библиотеки jQuery. Несмотря на то, что данное решение функционировало в течение длительного времени и выполняло основные требования, его дальнейшая эксплуатация и развитие сопряжены с рядом серьёзных технических ограничений.

Первая проблема связана со сложностью управления DOM-деревом при масштабировании интерфейса. В jQuery-приложении разработчик вручную манипулирует HTML-элементами: показывает и скрывает блоки, обновляет содержимое, добавляет и удаляет классы. При небольшом количестве элементов такой подход вполне управляем. Однако интерфейс колл-центра является принципиально динамичным объектом: на экране одновременно отображаются списки активных вызовов, панели управления операторами, блоки статусов, история звонков и уведомления. Каждый из этих элементов может изменяться независимо и асинхронно.

Вторая проблема обусловлена требованиями к обновлению данных в режиме реального времени. Интерфейс оператора колл-центра должен немедленно реагировать на внешние события: поступление нового звонка, изменение статуса коллеги, завершение вызова. В jQuery-архитектуре обработка каждого такого события требует написания явного кода обновления DOM. Это не только увеличивает объём кода, но и повышает вероятность ошибок рассинхронизации между реальным состоянием данных и тем, что отображается на экране.

Третья проблема — отсутствие компонентного подхода. Современная разработка фронтенда предполагает разделение интерфейса на независимые переиспользуемые компоненты, каждый из которых инкапсулирует собственную логику отображения и взаимодействия. В jQuery подобная изоляция крайне затруднена: код, как правило, распределён по обработчикам событий и функциям обновления без чёткой структурной организации. Это делает рефакторинг, тестирование и переиспользование компонентов практически невозможными.

Четвёртая проблема — отсутствие статической типизации. jQuery-приложения традиционно пишутся на чистом JavaScript, который является динамически типизированным языком. В крупных проектах это существенно увеличивает количество ошибок, обнаруживаемых только в процессе выполнения. Использование TypeScript позволяет переводить значительную часть ошибок в категорию компиляционных, повышая надёжность кода.

Совокупность перечисленных факторов однозначно свидетельствует о необходимости переработки веб-клиента с переходом на современный технологический стек. Новая реализация должна устранить все выявленные недостатки, сохранив при этом полную функциональную совместимость с серверной частью на базе FreeSWITCH.

1.5. Постановка задачи и требования к системе

На основе проведённого анализа предметной области, изучения существующих решений и выявленных недостатков исходного веб-клиента сформулируем полный перечень требований к разрабатываемой системе.

Функциональные требования определяют, что система должна уметь делать. Разрабатываемый веб-клиент обязан обеспечивать следующие возможности:

- 1) авторизация оператора в системе с использованием учётных данных и хранением сессии;

- 2) отображение списка всех операторов системы с их текущими статусами в режиме реального времени;
- 3) управление собственным статусом оператора (доступен, занят, недоступен и другие состояния);
- 4) приём входящего вызова с отображением информации о звонящем;
- 5) отклонение входящего вызова;
- 6) совершение исходящего вызова на внутренний номер или внешний телефон;
- 7) завершение активного вызова;
- 8) перевод вызова на другого оператора или номер;
- 9) отображение списка активных вызовов в системе;
- 10) просмотр истории вызовов (CDR) с информацией о времени, длительности и направлении;
- 11) передача голосового трафика непосредственно через браузер с использованием WebRTC;
- 12) мгновенное обновление интерфейса при получении событий от сервера через WebSocket.

Нефункциональные требования определяют качественные характеристики системы. К ним относятся следующие:

- 1) масштабируемость архитектуры — возможность добавления новых функций без переработки существующей кодовой базы;
- 2) удобство сопровождения — чёткое разделение кода по слоям и компонентам, соответствие стандартам Feature-Sliced Design;
- 3) безопасность типов — использование TypeScript для минимизации ошибок на этапе разработки;
- 4) производительность — отзывчивость интерфейса при обработке множественных событий в реальном времени;
- 5) совместимость — работа в современных браузерах (Chrome, Firefox, Edge последних версий) без дополнительных расширений;

- 6) безопасность — защита сессии, валидация входящих данных, использование HTTPS и WSS при развёртывании в продуктивной среде.

При разработке системы приняты следующие ограничения: серверная часть реализована на FreeSWITCH и не является предметом данной курсовой работы; взаимодействие с сервером осуществляется через заранее определённый API (HTTP REST и WebSocket); использование STUN-сервера включается или отключается пользователем через настройки приложения; разработка и тестирование проводятся в локальной сетевой среде.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Архитектура приложения (Feature-Sliced Design)

Выбор архитектурного подхода является одним из наиболее важных решений при проектировании фронтенд-приложения. От правильно выбранной методологии зависят масштабируемость проекта, удобство его сопровождения, скорость внесения изменений и понятность кода для новых участников команды. В данной работе используется методология Feature-Sliced Design (FSD) — архитектурный стандарт, специально разработанный для организации масштабируемых фронтенд-приложений.

Feature-Sliced Design — это открытая методология структурирования фронтенд-кода, основанная на принципах высокой связности и низкой сцепленности. Она определяет иерархию из нескольких слоёв (layers), каждый из которых выполняет строго определённую роль. Принципиально важным правилом является то, что модули верхних слоёв могут импортировать только из нижележащих, но не наоборот. Это обеспечивает однонаправленный поток зависимостей и предотвращает возникновение циклических импортов.

В разрабатываемом приложении используются следующие слои FSD:

- app — Верхний слой приложения. Здесь располагаются провайдеры (Redux Store, Router), глобальные стили и точка входа приложения. Именно в этом слое настраивается общая структура и инициализируются все глобальные сервисы.

- pages — Слой страниц. Каждая страница представляет собой отдельный маршрут приложения. В данном проекте выделены страницы авторизации (LoginPage) и основная рабочая страница оператора (MainPage). Страницы komponуют виджеты и фичи, не содержа собственной бизнес-логики.

- widgets — Слой виджетов содержит крупные самодостаточные блоки интерфейса, объединяющие несколько фичей. Примером является панель

управления вызовами, которая включает в себя элементы управления активным звонком и блок входящего вызова.

- features — Слой фич реализует конкретные пользовательские сценарии: авторизацию, совершение исходящего звонка, изменение статуса, перевод вызова. Каждая фича изолирована и не знает о существовании других фич.

- entities — Слой сущностей содержит бизнес-объекты системы и логику работы с ними. Выделены сущности: User (оператор), Session (сессия), Bundle (активное соединение), VertoCall (WebRTC-вызов), CDR (запись о вызове).

- shared — Нижний слой, содержащий переиспользуемые утилиты, базовые UI-компоненты (кнопки, иконки, поля ввода), константы, типы TypeScript, хелперы для работы с API и конфигурационные файлы. Этот слой не зависит ни от одного другого слоя проекта.

Помимо слоёв, FSD вводит понятие слайсов — подразделений внутри слоя по бизнес-домену (например, слайс user внутри слоя entities), и сегментов — технического разбиения внутри слайса на папки ui/, model/, api/, lib/, config/. Такая трёхуровневая организация (слой → слайс → сегмент) обеспечивает исключительную предсказуемость структуры проекта.

Использование FSD в данном проекте обосновано несколькими аргументами. Во-первых, интерфейс оператора АТС является функционально насыщенным приложением с множеством независимых сценариев. Во-вторых, методология обеспечивает возможность параллельной разработки разных частей системы. В-третьих, чёткие правила импортов устраняют архитектурные ошибки ещё на этапе проектирования.

2.2. Выбор и обоснование стека технологий

Выбор технологического стека оказывает определяющее влияние на все характеристики разрабатываемого приложения: производительность, надёжность, удобство разработки и долгосрочную сопровождаемость.

Для построения пользовательского интерфейса выбрана библиотека React (версия 18). React — это декларативная JavaScript-библиотека для построения пользовательских интерфейсов, разработанная компанией Meta. Её ключевыми преимуществами являются: компонентная модель, позволяющая разбить интерфейс на изолированные переиспользуемые блоки; виртуальный DOM, обеспечивающий эффективное обновление только изменившихся частей интерфейса; однонаправленный поток данных, упрощающий отладку; богатая экосистема библиотек и инструментов.

В сравнении с альтернативами — Vue.js и Angular — React предоставляет наибольшую гибкость при выборе вспомогательных библиотек, имеет наибольшее сообщество разработчиков и лучшую совместимость с экосистемой Redux. Angular, несмотря на свою зрелость, является избыточно сложным для данного проекта и накладывает жёсткие архитектурные ограничения. Vue.js является хорошей альтернативой, однако экосистема управления состоянием (Pinia/Vuex) менее развита по сравнению с Redux Toolkit.

Весь код приложения написан на TypeScript — надмножестве JavaScript, добавляющем статическую типизацию. Применение TypeScript позволяет достичь нескольких важных целей: обнаруживать ошибки на этапе компиляции, а не в процессе выполнения; получать подсказки автодополнения в IDE, значительно ускоряющие разработку; документировать интерфейсы функций и компонентов через описание типов; упрощать рефакторинг за счёт автоматической проверки корректности изменений.

Для проекта АТС-клиента типизация особенно важна: все объекты, передаваемые через WebSocket и HTTP API, а также структуры данных Redux-хранилища описаны строгими TypeScript-интерфейсами. Это позволяет гарантировать корректность обработки событий от сервера и предотвращает целый класс ошибок, связанных с несоответствием форматов данных.

Управление состоянием приложения реализовано с помощью Redux Toolkit (RTK) — официальной надстройки над библиотекой Redux,

устраняющей основные проблемы классического Redux: избыточность шаблонного кода, сложность настройки и отсутствие встроенной поддержки иммутабельности. RTK предоставляет функции `createSlice` и `createAsyncThunk`, значительно сокращающие объём кода при сохранении всех преимуществ централизованного хранилища.

Выбор Redux обусловлен характером разрабатываемого приложения: состояние АТС-клиента является глобальным — данные о вызовах, операторах и сессии требуются одновременно в разных частях интерфейса. Попытка управлять этим состоянием через локальные `useState`-хуки React привела бы к «prop drilling» — передаче данных через многоуровневые цепочки компонентов. Redux решает эту проблему за счёт централизованного хранилища, доступного из любого компонента.

Для выполнения HTTP-запросов к серверному API использована библиотека RTK Query, входящая в состав Redux Toolkit. RTK Query предоставляет декларативный подход к описанию запросов: разработчик описывает эндпоинты (авторизация, получение списка пользователей, получение CDR), а RTK Query автоматически генерирует React-хуки для их вызова, управляет кэшированием ответов, обрабатывает состояния загрузки и ошибок. Это позволяет полностью устранить шаблонный код по управлению асинхронными запросами.

Маршрутизация внутри приложения реализована посредством библиотеки React Router v6. Она обеспечивает декларативное описание маршрутов и навигацию без перезагрузки страницы (SPA-навигация). В данном приложении определены два основных маршрута: страница авторизации (/) и основная рабочая страница оператора (/main), доступная только после успешного входа в систему.

В качестве инструмента сборки и запуска проекта выбран Vite — современный сборщик нового поколения, разработанный Эваном Ю (создателем Vue.js). По сравнению с Webpack, традиционно используемым в экосистеме React (через Create React App), Vite демонстрирует значительно

более высокую скорость как при холодном старте, так и при пересборке после изменений.

Для взаимодействия с FreeSWITCH приложение использует три канала коммуникации. HTTP REST обеспечивает операции, не требующие реального времени: авторизацию, загрузку начального состояния системы и выполнение команд управления. WebSocket поддерживает постоянное двунаправленное соединение с сервером для получения событий в реальном времени — изменений статусов операторов, поступления вызовов, обновления списка активных соединений. WebRTC через протокол Verto обеспечивает непосредственную передачу голосового трафика между браузером оператора и сервером FreeSWITCH, используя стандарт SRTP для шифрования медиапотока.

2.3. Проектирование компонентной структуры

На уровне pages определены два корневых маршрута приложения. LoginPage отвечает за отображение формы авторизации и обработку процесса входа в систему. MainPage является основной рабочей страницей оператора и компоует все ключевые виджеты интерфейса.

На уровне widgets выделены следующие крупные блоки интерфейса:

- UsersPanel — панель со списком всех операторов системы, отображающая их имена, внутренние номера и текущие статусы. Обновляется в реальном времени при получении событий статуса через WebSocket.
- Header — центральная панель управления вызовами. Отображает активный звонок, кнопки управления (завершить, перевести, удержать) и блок входящего вызова с возможностью принятия или отклонения.
- ActiveCallsTable — блок активных вызовов в системе, показывающий все текущие соединения с указанием участников, типа вызова и его длительности.
- CallHistoryPanel — компонент истории вызовов, отображающий записи CDR с фильтрацией по дате и типу вызова.

- `StatusDropdown` — виджет управления собственным статусом оператора с выпадающим меню доступных состояний.

На уровне `features` реализованы следующие изолированные пользовательские сценарии:

- `auth` — логика авторизации: форма входа, валидация полей, отправка запроса на сервер, сохранение данных сессии в `Redux Store`.

- `makeCall` — функциональность совершения исходящего звонка: поле ввода номера, инициация вызова через `VertoClient`.

- `answerCall` — приём входящего вызова: обработка события поступления звонка, отображение уведомления, подключение к вызову.

- `transferCall` — перевод активного вызова: выбор целевого оператора или ввод номера, выполнение команды перевода.

- `changeStatus` — изменение рабочего статуса оператора: выбор нового статуса, отправка обновления на сервер.

- `hangupCall` — завершение активного вызова с корректным закрытием WebRTC-соединения.

На уровне `entities` определены бизнес-объекты с их `Redux`-слайсами и `TypeScript`-интерфейсами:

- `User` — описывает оператора системы: идентификатор, имя, внутренний номер, текущий статус. Содержит селекторы для получения списка операторов и поиска по номеру.

- `Session` — хранит данные текущей сессии: идентификатор сессии (`sessionID`), имя и идентификатор пользователя, текущий статус, адрес `Verto`-сервера и состояние `WebSocket`-соединения.

- `Bundle` — представляет активное телефонное соединение: идентификатор звонка, направление (входящий/исходящий), участники, состояние (`ringing`, `active`, `held`).

- `VertoCall` — обёртка над WebRTC-соединением: управляет состоянием медиапотока, ICE-кандидатами, SDP-обменом через протокол `Verto`.

– CDR — запись детальной информации о завершённом вызове: время начала и окончания, длительность, направление, номера участников.

На уровне shared собраны переиспользуемые элементы: базовые UI-компоненты (Button, Input, Modal, Badge), утилиты форматирования времени и номеров, константы состояний операторов и вызовов, конфигурация API-эндпоинтов, а также TypeScript-типы, используемые в нескольких слоях.

Описанная компонентная структура обеспечивает чёткое разделение ответственности, где каждый компонент решает строго одну задачу, что соответствует принципу единственной ответственности (Single Responsibility Principle) из набора принципов SOLID.

2.4. Проектирование взаимодействия с сервером

Взаимодействие клиентского приложения с серверной частью на базе FreeSWITCH реализовано через три независимых канала коммуникации, каждый из которых оптимизирован для своего класса задач.

Для выполнения HTTP-запросов используется ApiSlice, построенный на основе RTK Query. Базовый URL сервера вынесен в конфигурационный файл и может быть изменён при развёртывании. Все запросы отправляются с параметром credentials: 'include', что обеспечивает автоматическую передачу cookie авторизации.

Через HTTP API реализованы следующие операции: авторизация (метод Authenticate), завершение сессии (Terminate), получение списка пользователей (User.Search), получение состояний пользователей (UserState.Search), поиск групп (CallGroup.Search), получение активных соединений (BundleState.Search), управление вызовами (Call.Hangup, Call.Transfer, Call.ConsultTransfer), поиск истории вызовов (CDR.Search).

RTK Query автоматически управляет кэшированием ответов: данные о списке операторов кэшируются и обновляются только при явном запросе или

при получении соответствующего события через WebSocket. Это позволяет минимизировать нагрузку на сервер и избежать лишних запросов.

Для получения событий в реальном времени реализован модуль WsClient, управляющий WebSocket-соединением с сервером. Соединение устанавливается сразу после успешной авторизации и поддерживается в течение всей рабочей сессии оператора.

WsClient реализует механизм автоматического переподключения при разрыве соединения: при потере связи модуль выполняет последовательные попытки восстановления с экспоненциально возрастающим интервалом ожидания (1 с, 2 с, 4 с, 8 с, максимум 30 с). Это обеспечивает устойчивость интерфейса при кратковременных сетевых сбоях.

Все входящие события от сервера принимаются модулем WsClient и маршрутизируются функцией routeWsMessage, которая на основе типа события диспатчит соответствующие действия в Redux Store. Обработываются следующие события: обновление данных пользователя, изменение сетевого статуса, обновление и завершение активных соединений, изменения в группах и их агентах, а также появление новой записи в истории вызовов.

Голосовая связь реализована через модуль VertoClient, обеспечивающий интеграцию с FreeSWITCH по протоколу Verto. Verto работает поверх отдельного WebSocket-соединения и использует формат JSON-RPC 2.0 для передачи управляющих сообщений.

Процесс установления WebRTC-соединения включает несколько этапов. При инициации исходящего вызова VertoClient создаёт объект RTCPeerConnection, формирует SDP-offer (Session Description Protocol), содержащий описание медиавозможностей браузера, и отправляет его серверу через Verto-сообщение verto.invite. После завершения ICE-обмена (Interactive Connectivity Establishment) медиапоток начинает передаваться напрямую между браузером и FreeSWITCH.

Для прохождения NAT используется публичный STUN-сервер Google (stun:stun.l.google.com:19302), заданный в коде по умолчанию. Пользователь

может включить или отключить использование STUN через чекбокс в настройках приложения — значение сохраняется в `localStorage` под ключом `settings.useStun`.

2.5. Модель данных и описание основных сущностей

Модель данных приложения описывает структуру объектов, хранящихся в `Redux Store`, а также форматы данных, передаваемых через `API` и `WebSocket`. Все типы данных описаны средствами `TypeScript`, что обеспечивает их строгую валидацию на этапе компиляции.

Центральным элементом архитектуры является `Redux Store` — единое хранилище состояния приложения. `Store` объединяет несколько независимых слайсов, каждый из которых управляет данными своего домена. Рассмотрим основные сущности системы.

Сущность `Session` хранит данные текущей авторизованной сессии оператора. Она включает следующие поля: `sessionID` — идентификатор сессии; `userName` и `userId` — данные авторизованного пользователя; `availStatus` — текущий статус оператора; `vertoUrl` — адрес Verto-сервера; `useVerto` — флаг использования `WebRTC`; `wsStatus` — состояние `WebSocket`-соединения. Слайс `sessionSlice` обрабатывает actions `setSession`, `clearSession`, `setAvailStatus` и `setWsStatus`.

Сущность `User` описывает оператора телефонной системы и содержит поля: `id` — уникальный идентификатор; `name` — техническое имя пользователя; `commonName` — отображаемое имя; `numbers` — массив телефонных номеров; `availStatus` и `busyStatus` — составной статус оператора. Отдельно хранится объект `UserState` с динамическими данными: `networkStatus` — статус сетевого подключения; `busyCount` — количество активных вызовов. Разделение статических данных пользователя (`User`) и его динамического состояния (`UserState`) позволяет эффективно обновлять только изменившуюся часть без перерисовки всего компонента.

Сущность Bundle представляет активное телефонное соединение в системе. Объект Bundle включает: id — уникальный идентификатор соединения; domainID — идентификатор домена; services — массив сервисов типа BundleService. Каждый сервис содержит поля connState — состояние соединения, cgpn и cdpn — номера звонящего и вызываемого, поля caller и callee с именами участников, а также временные метки createdTime, answeredTime, hangupTime. Слайс bundleSlice содержит коллекцию всех активных Bundle-объектов и обновляется при получении событий VirtaEvent.BundleState.Updated через WebSocket.

Сущность VertoCall является прослойкой между Redux и WebRTC API браузера. Она хранит: callID — уникальный идентификатор вызова; direction — направление (inbound/outbound); destinationNumber, callerIdName, callerIdNumber — данные участников; state — состояние вызова (new, trying, ringing, early, active, held, hangup, destroy); startedAt и answeredAt — временные метки. Объекты RTCPeerConnection не сериализуются и хранятся отдельно в Map activeSessions внутри модуля webrtcManager.ts.

Сущность CDR (Call Detail Record) содержит информацию о завершённых вызовах и используется для отображения истории звонков. Поля сущности: id — уникальный идентификатор записи; createdTime, answeredTime, hangupTime — временные метки в виде числового timestamp; cgpn и cdpn — номера звонящего и вызываемого; caller.commonName, caller.commonNumber — отображаемые данные звонящего; callee.commonName, callee.commonNumber — данные вызываемого; type — тип вызова; connState — состояние соединения; hangupSide — сторона, завершившая вызов. CDR-записи загружаются через JSON-RPC метод CDR.Search с поддержкой пагинации и фильтрации по дате, результаты кэшируются в Redux-слайсе cdrSlice.

Время вызова	Номер	Имя	Тип звонка	Длительность
21:29:26 02 мая	103	Девелопер 3	Входящий	0:00:09
21:27:35 02 мая	103	Девелопер 3	Входящий	0:00:43
09:25:32 28 апреля	102	Девелопер 2	Внутренний	
09:25:21 28 апреля	102	Девелопер 2	Внутренний	
09:25:11 28 апреля	102	Девелопер 2	Внутренний	
14:33:36 22 апреля	102	Девелопер 2	Входящий	0:00:51
12:58:45 22 апреля	102	Девелопер 2	Входящий	
12:57:59 22 апреля	102	Девелопер 2	Входящий	
12:56:51 22 апреля	102	Девелопер 2	Исходящий	0:00:05
07:59:41 22 апреля	103	Девелопер 3	Исходящий	
07:59:07 22 апреля	102	Девелопер 2	Исходящий	0:00:17
23:12:54 21 апреля	102	Девелопер 2	Исходящий	0:00:31
08:26:16 21 апреля	102	Девелопер 2	Исходящий	0:00:15
08:25:50 21 апреля	102	Девелопер 2	Исходящий	
13:25:33 20 апреля	103	Девелопер 3	Исходящий	0:00:08
12:16:02 20 апреля	103	Девелопер 3	Исходящий	0:00:33
00:03:04 20 апреля	103	Девелопер 3	Входящий	
15:38:26 19 апреля	103	Девелопер 3	Входящий	0:01:35
15:37:19 19 апреля	103	Девелопер 3	Входящий	0:00:05
10:18:52 18 апреля	102	Девелопер 2	Входящий	
21:59:25 11 апреля	102	Девелопер 2	Исходящий	
20:47:54 11 апреля	102	Девелопер 2	Исходящий	
12:59:20 10 апреля	103	Девелопер 3	Входящий	0:00:11
12:58:33 10 апреля	103	Девелопер 3	Входящий	
12:53:40 10 апреля	103	Девелопер 3	Исходящий	

Рисунок 1- История вызовов

Описанная модель данных обеспечивает полное отображение состояния телефонной системы в клиентском приложении. Централизованное хранение в Redux Store гарантирует согласованность данных между всеми компонентами интерфейса и упрощает отладку с помощью Redux DevTools.

ГЛАВА 3. РЕАЛИЗАЦИЯ ВЕБ-КЛИЕНТА

3.1. Структура проекта и слои FSD

Реализация приложения начинается с формирования корректной файловой структуры проекта в соответствии с методологией Feature-Sliced Design. Правильно организованная структура является фундаментом всей дальнейшей разработки: она определяет, где искать тот или иной код, как модули связаны между собой и каковы границы ответственности каждой части системы.

Инициализация проекта выполняется с помощью Vite с шаблоном React и TypeScript. Эта команда создаёт базовую структуру с настроенными конфигурационными файлами: `vite.config.ts` для параметров сборки, `tsconfig.json` для настройки компилятора TypeScript и `package.json` со списком зависимостей.

Каждый слайс (например, `entities/user/`) содержит строго определённый набор сегментов. Сегмент `model/` содержит Redux-слайс, TypeScript-типы и селекторы. Сегмент `ui/` содержит React-компоненты, отвечающие за отображение данной сущности. Сегмент `api/` содержит RTK Query-эндпоинты для взаимодействия с сервером. Файл `index.ts` является публичным API слайса: он явно экспортирует только то, что должно быть видно снаружи. Доступ к внутренним файлам слайса в обход `index.ts` запрещён правилами FSD.

Для автоматической проверки соблюдения правил импортов FSD в проект добавлен плагин `eslint-plugin-boundaries`, настроенный на запрет импортов из вышестоящих слоёв и прямых импортов в обход публичных API. Это позволяет обнаруживать архитектурные нарушения ещё на этапе написания кода.

Настройка алиасов путей в `tsconfig.json` и `vite.config.ts` обеспечивает удобные абсолютные импорты вида `@/shared/ui/Button` вместо относительных `../../shared/ui/Button`, что повышает читаемость кода и упрощает рефакторинг при перемещении файлов.

3.2. Реализация авторизации и управления сессией

Авторизация является первым экраном, с которым взаимодействует оператор при запуске приложения, и одновременно критически важным модулем с точки зрения безопасности. Реализация авторизации охватывает несколько слоёв приложения: UI-форму, бизнес-логику фичи, HTTP-запрос через RTK Query и управление состоянием сессии через Redux.

Redux-слайс `sessionSlice` реализован с использованием `createSlice` из Redux Toolkit. Начальное состояние содержит поля: `sessionID` — идентификатор текущей сессии, `userName` и `userId` — данные авторизованного пользователя, `availStatus` — текущий статус оператора, `vertoUrl` — адрес Verto-сервера, `useVerto` — флаг использования WebRTC, `wsStatus` — состояние WebSocket-соединения. Авторизованность пользователя определяется наличием непустого поля `sessionID` (листинг A.1).

HTTP-запрос авторизации описан в `authApi` посредством RTK Query. Мутация `loginMutation` принимает объект с полями `login` и `password`, отправляет JSON-RPC запрос с методом `Authenticate` и возвращает ответ сервера, содержащий `sessionID` и данные пользователя. После успешного ответа обработчик `onQueryStarted` диспатчит action `setSession`, сохраняя данные в Redux Store.

Компонент `LoginPage` реализован как React-функциональный компонент с использованием хука `useLoginMutation` из сгенерированных RTK Query-хуков. Форма содержит два поля ввода (имя пользователя и пароль) и кнопку входа. При отправке формы вызывается мутация, а результат обрабатывается через механизм `unwrap()`, позволяющий отлавливать ошибки через стандартный `try-catch`. В случае ошибки авторизации пользователю отображается соответствующее сообщение.

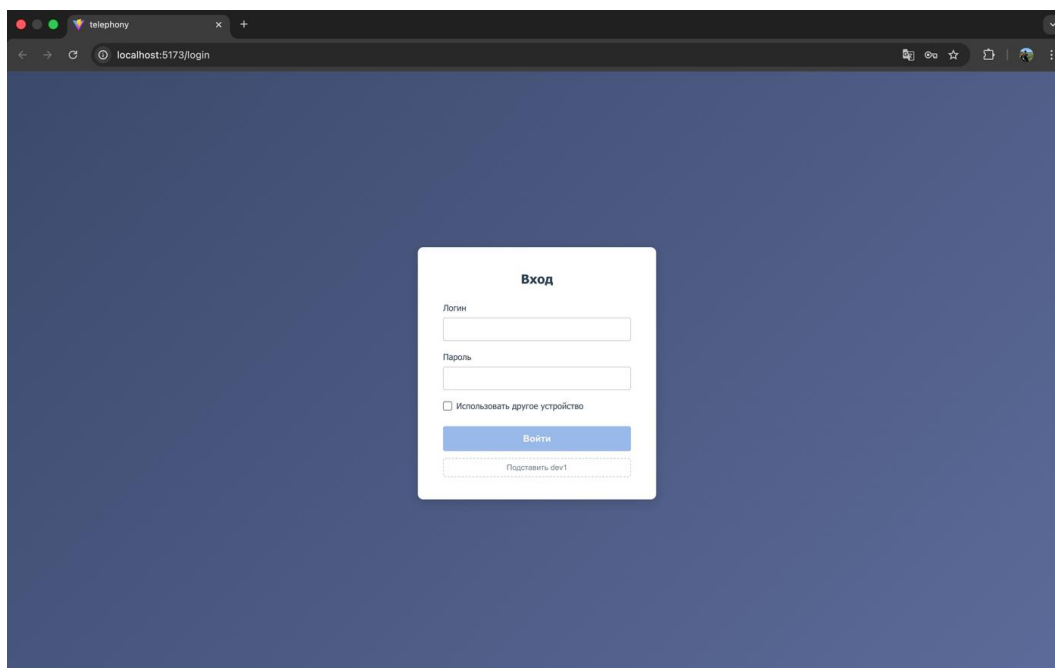


Рисунок 2 - Страница авторизации веб-клиента

Защищённая маршрутизация реализована через компонент `RequireAuth`, оборачивающий основной маршрут `/main`. Он считывает значение `selectIsAuthed` из `Redux Store`, которое возвращает `true` при наличии непустого поля `sessionID`. При перезагрузке страницы `Redux Store` сбрасывается, и пользователь перенаправляется на страницу входа.

При выходе из системы выполняется комплексная очистка состояния: диспатч action `clearSession` сбрасывает `Redux Store`, закрывается `WebSocket`-соединение через вызов `wsClient.disconnect()`, завершается `WebRTC`-сессия через `vertoClient.disconnect()` и `destroyAllSessions()`. Такой подход гарантирует полную очистку всех ресурсов при завершении рабочей сессии оператора.

3.3. Реализация отображения операторов и их статусов

Отображение списка операторов и их актуальных статусов является одним из ключевых элементов интерфейса оператора колл-центра. Супервизор или оператор должны в любой момент видеть, кто из коллег доступен для перевода вызова, кто занят, а кто находится вне рабочего места.

Реализация этой функциональности задействует все три канала взаимодействия с сервером.

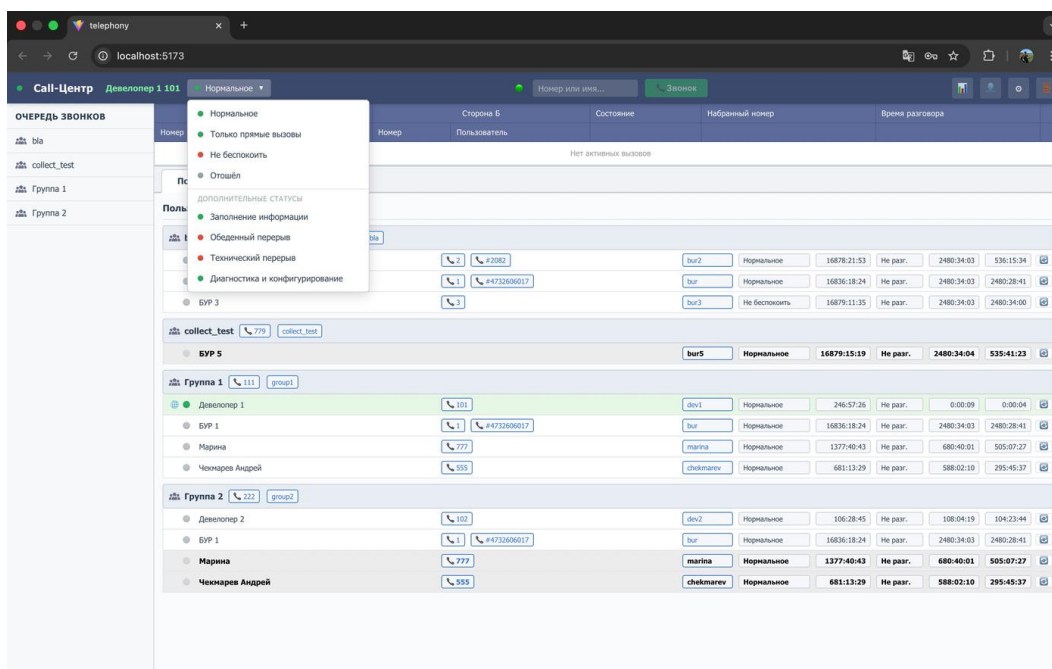


Рисунок 3 - Управление статусом оператора

Redux-слайс `userSlice` хранит коллекцию операторов в нормализованном виде вручную: данные операторов хранятся в объекте `entities` с ключами-идентификаторами и массиве `ids` для сохранения порядка. Отдельно хранится объект `states` — словарь динамических состояний каждого оператора (`networkStatus`, `busyCount`). Такая структура обеспечивает эффективное точечное обновление данных без пересоздания всей коллекции.

Начальная загрузка данных выполняется при монтировании основной страницы через хук `useBootstrap`, который параллельно запрашивает список пользователей (`users`), их состояния (`userStates`), группы, агентов и активные Bundle-соединения. После получения ответов данные диспатчатся в соответствующие Redux-слайсы.

Обновление статусов в реальном времени реализовано через `WsClient`.

При получении события `VirtaEvent.UserState.Updated`, содержащего идентификатор оператора и его новый статус, диспатчится action `updateUserState`, который обновляет соответствующую запись в `userSlice` без перезагрузки всего списка.

Виджет `UsersPanel` реализован как компонент, подписанный на селектор `selectRoster`, который формирует отсортированный список групп и агентов. Список операторов отсортирован по приоритету: сначала отображаются доступные операторы, затем занятые, затем остальные. Каждый элемент списка представляет собой компонент `AgentRowItem`, отображающий имя оператора, его номер и цветовой индикатор статуса (листинг A.2).

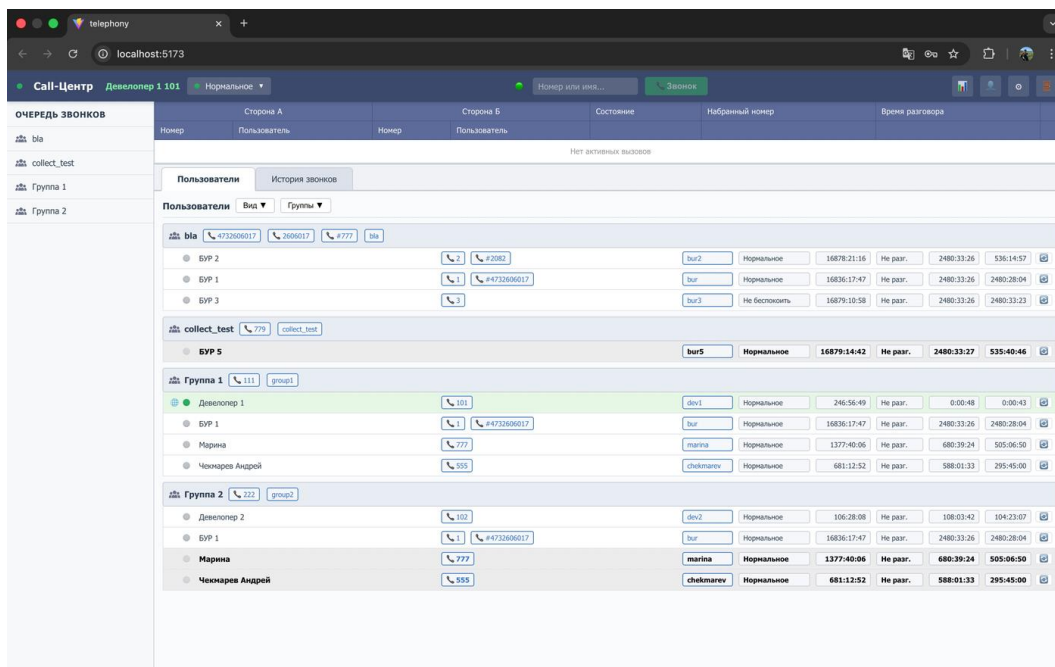


Рисунок 4 - Главная страница

Для управления собственным статусом реализован виджет `StatusDropdown`. Он отображает выпадающее меню с доступными состояниями (доступен, занят, не беспокоить, отошёл) и текущим выбранным статусом. Это обеспечивает мгновенную визуальную реакцию интерфейса на действие пользователя.

Реализованная система отображения операторов обеспечивает актуальность данных без необходимости ручного обновления страницы, что является критически важным для операторов колл-центра при принятии решений о переводе вызовов.

3.4. Реализация обработки входящих и исходящих звонков

Обработка вызовов является центральной функциональностью всего приложения. Реализация данного модуля охватывает управление состоянием звонков в Redux, обработку событий от сервера через WebSocket и управление жизненным циклом WebRTC-соединения через VertoClient.

Состояние активных вызовов хранится в bundleSlice. Слайс содержит объекты Bundle в виде словаря entities и массива ids. Предусмотрены редьюсеры setAll (полная замена коллекции), upsertBundle (добавление или обновление одного Bundle) и removeBundle (удаление по идентификатору).

Обработка входящего вызова начинается с получения события VirtaEvent.BundleState.Updated через WsClient. Обработчик события создаёт объект Bundle на основе данных события и добавляет его в bundleSlice. Одновременно обработчик onCallCreated в vertoMiddleware запускает рингтон через Web Audio API.

Компонент IncomingCallNotification отображается при наличии Bundle с состоянием ringing. Он показывает имя и номер звонящего, а также две кнопки: принять и отклонить вызов (листинг A.3).

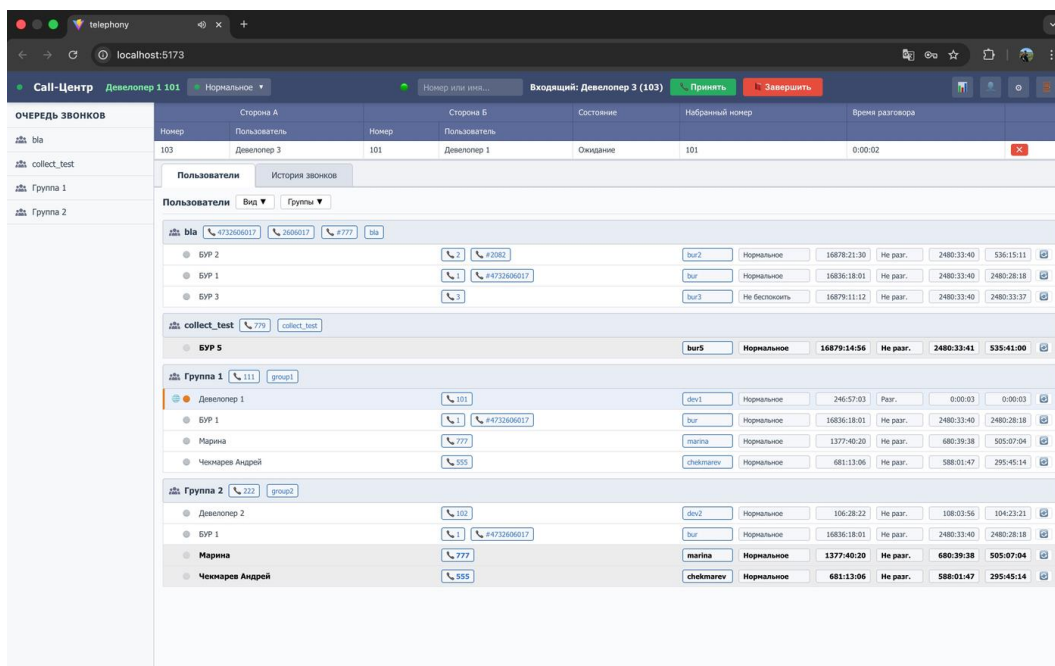
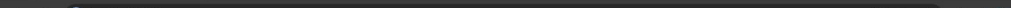


Рисунок 5 - Уведомление о входящем вызове

При нажатии кнопки «Принять» выполняется следующая последовательность действий: запрашивается доступ к микрофону пользователя; вызывается метод `vertClient.answer()`, который завершает SDP-обмен и устанавливает медиасоединение; состояние Bundle в Redux обновляется с `ringing` на `active`; компонент `IncomingCallNotification` скрывается, а `Header` переходит в режим активного звонка. При нажатии «Отклонить» вызывается `vertClient.bye()`, что отправляет Verto-сообщение `vertClient.bye` серверу и удаляет Bundle из хранилища.

Инициация исходящего вызова реализована через фичу `makeCall`. При нажатии кнопки создаётся `RTCPeerConnection`, формируется SDP-offer и отправляется серверу через `vertClient.invite()`. Одновременно в `bundleSlice` создаётся новый Bundle с направлением `outbound` и состоянием `ringing`. Нажатие на имя или номер оператора в компоненте `AgentRowItem` автоматически подставляет его номер.

Активный звонок отображается в компоненте `ActiveCallsTable`, который показывает: имя и номер собеседника, таймер длительности текущего вызова



A screenshot of a web browser window. The address bar shows the URL "telephony". The page content is mostly black, with some faint, illegible text visible in the background.

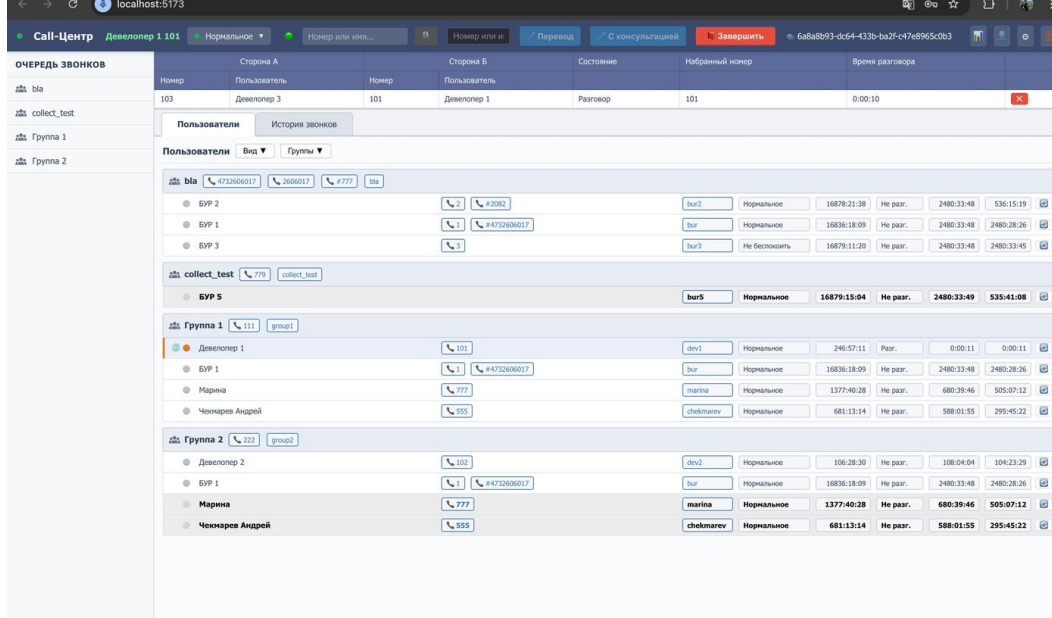


Рисунок 6 - Панель управления активным вызовом

Перевод вызова реализован непосредственно в компоненте Header. При наличии активного звонка в панели появляются поле ввода номера или имени получателя и две кнопки: «Перевод» (метод `blindTransfer`) и «С консультацией» (метод `attendedTransfer`). После нажатия «С консультацией» панель переходит в режим консультации с кнопкой «Назад» для возврата к исходному звонку.

Завершение вызова обрабатывается как по инициативе оператора (кнопка «Завершить»), так и при получении события `VirtaEvent.BundleState.Updated` или `VirtaEvent.BundleState.Destroyed` через `WebSocket`. В обоих случаях выполняется закрытие `RTCPeerConnection`, остановка всех медиатреков и удаление `Bundle` из хранилища.

3.5. Интеграция WebRTC через протокол Verto

Модуль `VertoClient` является наиболее технически сложным компонентом всей системы, поскольку реализует взаимодействие на стыке двух технологий: протокола Verto для управляющей сигнализации и WebRTC для передачи медиаданных.

`VertoClient` реализован как класс, единственный экземпляр которого создаётся при инициализации модуля и экспортируется как константа `vertClient`. Подключение к серверу выполняется через `Redux middleware (vertMiddleware)` сразу после успешной авторизации: `middleware` перехватывает `action setSession`, извлекает из него `vertUrl` и иницирует `WebSocket-соединение с FreeSWITCH (листинг A.4)`.

Обработка входящих Verto-сообщений реализована в методе `handleMessage()`. Метод разбирает `JSON-RPC` объект и диспатчит вызов соответствующего обработчика в зависимости от метода: `vert.invite` (входящий вызов), `vert.answer` (ответ на исходящий), `vert.bye` (завершение), `vert.media` (обмен `SDP`) и `vert.display` (обновление данных собеседника). Каждый обработчик также диспатчит соответствующий `Redux-action` для обновления состояния приложения.

Процесс установления исходящего WebRTC-соединения реализован в методе `invite()` и включает несколько этапов. На первом этапе запрашивается доступ к микрофону через `getUserMedia()` и создаётся объект `RTCPeerConnection` с конфигурацией ICE-серверов, включающей `STUN-сервер` из настроек пользователя. На втором этапе локальный медиапоток добавляется в соединение через `addTrack()`, что иницирует процесс ICE-кандидатов. На третьем этапе вызывается `createOffer()` для формирования `SDP-offer`, после чего он устанавливается как `localDescription` и отправляется серверу `FreeSWITCH` в теле Verto-сообщения `vert.invite` (листинг A.5).

После получения от `FreeSWITCH` ответного Verto-сообщения `vert.answer` с `SDP-answer` `VertoClient` вызывает `setRemoteDescription()` на соответствующем `RTCPeerConnection`. С этого момента ICE-согласование

продолжается в фоновом режиме: браузер и сервер обмениваются ICE-кандидатами до установления оптимального медиапути. После завершения ICE-согласования аудиопоток начинает передаваться в обоих направлениях.

Воспроизведение входящего аудиопотока реализовано через HTML-элемент `<audio>` с атрибутом `autoplay`. Аудиоэлемент создаётся динамически в `webrtcManager.ts` через функцию `createRemoteAudioElement()` и хранится внутри объекта сессии `WebRtcCallSession`. Такой подход обеспечивает корректное воспроизведение аудио без необходимости взаимодействия пользователя с DOM.

Завершение WebRTC-сессии выполняется методом `bye()`. Он последовательно останавливает все медиатреки (предотвращая утечку ресурсов и отключая индикатор использования микрофона в браузере), закрывает `RTCPeerConnection`, удаляет запись из Map активных вызовов и отправляет FreeSWITCH сообщение `vertorbye` для корректного завершения на серверной стороне.

3.6. Система хранения данных

Система хранения данных в разрабатываемом приложении является многоуровневой: различные типы данных хранятся в разных хранилищах в зависимости от их характера, времени жизни и требований к доступности.

Основным хранилищем данных приложения является `Redux Store`, конфигурируемый в файле `app/store/store.ts` с помощью функции `configureStore` из `Redux Toolkit`.

`Store` объединяет несколько редьюсеров: `sessionReducer`, `userReducer`, `callGroupReducer`, `bundleReducer`, `cdReducer`, `vertorReducer`, `groupOrderReducer`, `viewSettingsReducer` (листинг A.6).

`Redux DevTools Extension` поддерживается автоматически благодаря `configureStore` из `RTK`. В режиме разработки это позволяет разработчику наблюдать все диспатчируемые `actions` в хронологическом порядке, инспектировать состояние `Store` в любой момент времени, «путешествовать во

времени» — откатываться к предыдущим состояниям для воспроизведения ошибок. Эта возможность оказалась особенно полезной при отладке асинхронных сценариев обработки вызовов.

Второй уровень хранения — `sessionStorage` браузера — используется для хранения Verto-credentials (логина и пароля для WebRTC-соединения) под ключом `_verto_creds`. Эти данные сохраняются при авторизации и удаляются при выходе из системы. Авторизационная сессия поддерживается на стороне сервера, а её наличие определяется по полю `sessionID` в Redux Store.

Третий уровень — `localStorage` — используется для хранения пользовательских настроек, которые должны сохраняться между сеансами работы. В текущей реализации в `localStorage` сохраняются: флаг использования STUN-сервера (`settings.useStun`), порядок групп в списке (`groupOrder.v1`), настройки отображения операторов (`viewSettings.v1`), а также настройки рингтона и звука завершения вызова. Управление настройками реализовано непосредственно в компоненте `SettingsModal`, который читает и записывает значения в `localStorage` (листинг А.7).

Кэширование данных HTTP-запросов управляется `RTK Query`. Инвалидация кэша происходит автоматически при диспатче соответствующих тегов `invalidatesTags`, что позволяет принудительно обновить данные при получении WebSocket-события.

Серверное хранение данных. Часть данных хранится исключительно на стороне сервера FreeSWITCH: CDR-записи о завершённых вызовах, информация о пользователях (имена, номера, пароли), история изменения статусов. Эти данные загружаются в Redux Store по запросу и обновляются при необходимости через `RTK Query`-эндпоинты. Клиентская часть не дублирует серверные данные без необходимости, ограничиваясь хранением только тех записей, которые актуально нужны для текущего состояния интерфейса.

Описанная многоуровневая система хранения данных оптимально распределяет информацию по хранилищам с учётом её природы: оперативные

данные о текущих вызовах и статусах — в Redux Store с немедленным обновлением через WebSocket; авторизационные данные — в sessionStorage с автоматическим истечением; пользовательские настройки — в localStorage с сохранением между сеансами; исторические данные — на сервере с кэшированием на клиенте через RTK Query.

ГЛАВА 4. ТЕСТИРОВАНИЕ И РЕЗУЛЬТАТЫ

4.1. Описание тестового стенда

Серверная часть на базе FreeSWITCH была предоставлена в готовом к работе виде и не являлась предметом настройки в рамках данной курсовой работы. Веб-клиент подключался к уже функционирующему серверу по заранее известным адресам и учётным данным.

Клиентская часть стенда представлена двумя рабочими местами. Тестирование проводилось в трёх браузерах: Google Chrome версии 124, Mozilla Firefox версии 125 и Microsoft Edge версии 124. В каждом браузере проверялась полная функциональность приложения.

Тестирование проводилось методом ручного функционального тестирования: оба тестировщика последовательно выполняли проверочные сценарии непосредственно в браузере, взаимодействуя с интерфейсом приложения и фиксируя фактический результат каждого сценария.

4.2. Функциональное тестирование

Функциональное тестирование проводилось по методу чёрного ящика: тестировщики взаимодействовали с интерфейсом приложения, не имея доступа к исходному коду, и проверяли соответствие поведения системы сформулированным в первой главе требованиям. Для каждого тест-кейса фиксировались: предусловие, шаги выполнения, ожидаемый результат и фактический результат.

Проверялись следующие сценарии: успешный вход в систему с корректными учётными данными, попытка входа с неверным паролем, попытка входа с несуществующим пользователем, корректный выход из системы с очисткой всех данных.

Тестирование входящих вызовов выполнялось путём совершения звонков с одного рабочего места оператора на другое через интерфейс веб-клиента. Проверялись отображение уведомления о входящем вызове,

корректность отображаемой информации о звонящем и работа кнопок управления.

Проверялось совершение исходящих вызовов как на внутренние номера операторов (через кнопку на карточке оператора), так и на произвольные номера через поле ввода. Тестировалось корректное установление WebRTC-соединения и двусторонняя передача аудио.

Проверялись операции с активным звонком: завершение вызова, постановка на удержание и снятие с удержания, перевод вызова на другого оператора.

Тестировалась корректность загрузки и отображения CDR-записей, работа фильтров и актуальность данных после завершения вызовов.

Каждый из ключевых сценариев (авторизация, приём и совершение вызова) был воспроизведён в трёх браузерах. Во всех случаях поведение приложения соответствовало ожидаемому. Незначительные различия в визуальном отображении, обусловленные особенностями рендеринга CSS в разных браузерах, не влияют на функциональность приложения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была достигнута поставленная цель: разработан современный, функциональный и масштабируемый веб-клиент для программной АТС на базе FreeSWITCH с использованием актуального стека технологий.

В процессе работы решены все сформулированные задачи. Проведён системный анализ предметной области, включающий изучение архитектуры FreeSWITCH и принципов взаимодействия с ним по протоколам HTTP, WebSocket и WebRTC (Verto); выполнен сравнительный анализ существующих решений и обоснован выбор технологического стека.

Спроектирована архитектура клиентского приложения с применением методологии Feature-Sliced Design, обеспечивающей чёткое разделение ответственности между слоями и возможность независимой разработки отдельных модулей. Разработана модель данных, описывающая все ключевые сущности системы (Session, User, Bundle, VertoCall, CDR) в виде строго типизированных TypeScript-интерфейсов.

Реализованы следующие модули: авторизация с защищёнными маршрутами и управлением сессией; отображение операторов с обновлением статусов в реальном времени; полный цикл обработки входящих и исходящих вызовов; модуль VertoClient для интеграции WebRTC с детальной обработкой всех этапов установления соединения; многоуровневая система хранения данных.

По результатам ручного функционального тестирования все тест-кейсы успешно пройдены. Приложение протестировано в трёх актуальных браузерах (Chrome, Firefox, Edge) и продемонстрировало полную кроссбраузерную совместимость. Все функциональные и нефункциональные требования выполнены.

Практическая ценность разработанного решения состоит в следующем. Во-первых, веб-клиент готов к использованию в реальной производственной среде в качестве интерфейса оператора колл-центра. Во-вторых, применённая

архитектура FSD и современный стек технологий (React, TypeScript, Redux Toolkit) обеспечивают долгосрочную сопровождаемость и масштабируемость системы. В-третьих, реализованная интеграция WebRTC исключает необходимость установки дополнительного программного обеспечения на рабочих местах операторов.

Таким образом, в ходе работы получен практически значимый результат: готовый к эксплуатации веб-клиент для программной АТС на базе FreeSWITCH, реализованный с соблюдением современных стандартов разработки и подтверждённый результатами тестирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дронов, В. А. React 17. Разработка веб-приложений на JavaScript / В. А. Дронов. — СПб. : БХВ-Петербург, 2022. — 384 с. — ISBN 978-5-9775-9683-1.
2. Розенталс, Н. Изучаем TypeScript 3 / Н. Розенталс ; пер. с англ. Д. А. Беликова. — М. : ДМК Пресс, 2019. — 608 с. — ISBN 978-5-97060-757-2.
3. Minessale II, A. FreeSWITCH 1.8 / A. Minessale II, G. Maruzzelli. — Birmingham : Packt Publishing, 2017. — 440 с. — ISBN 978-1-78588-913-4.
4. Minessale II, A. Mastering FreeSWITCH / A. Minessale II, G. Maruzzelli. — Birmingham : Packt Publishing, 2016. — 300 с. — ISBN 978-1-78439-888-0.
5. Flanagan, D. JavaScript: The Definitive Guide. Master the World's Most-Used Programming Language. 7th ed. / D. Flanagan. — Sebastopol : O'Reilly Media, 2020. — 706 с. — ISBN 978-1-491-95202-3.
6. Cherny, B. Programming TypeScript: Making Your JavaScript Applications Scale / B. Cherny. — Sebastopol : O'Reilly Media, 2019. — 322 с. — ISBN 978-1-492-03765-1.
7. Johnston, A. B. WebRTC: APIs and RTCWEB Protocols of the HTML5 Real-Time Web. 3rd ed. / A. B. Johnston, D. C. Burnett. — [S. l.] : Digital Codex LLC, 2014. — 350 с. — ISBN 978-0-9859788-6-0.
8. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures : дис. ... д-ра информатики / R. T. Fielding. — University of California, Irvine, 2000. — 180 с. — Режим доступа: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
9. Кантор, И. Современный учебник JavaScript [Электронный ресурс] / И. Кантор. — 2023. — Режим доступа: <https://learn.javascript.ru/> (дата обращения: 22.10.2025).
10. Feature Sliced Design — архитектура фронтенд-проектов [Электронный ресурс] // Habr.com. — 2022. — Режим доступа: <https://habr.com/ru/articles/733110/> (дата обращения: 22.10.2025).

11. WebRTC для начинающих [Электронный ресурс] // Habr.com. — 2020. — Режим доступа: <https://habr.com/ru/articles/489926/> (дата обращения: 28.10.2025).
12. Redux Toolkit: современный Redux без шаблонного кода [Электронный ресурс] // Habr.com. — 2021. — Режим доступа: <https://habr.com/ru/articles/591417/> (дата обращения: 05.11.2025).
13. FreeSWITCH. Official Documentation [Электронный ресурс] / SignalWire Inc. — Режим доступа: <https://developer.signalwire.com/freeswitch/> (дата обращения: 07.11.2025).
14. React. Documentation [Электронный ресурс] / Meta Open Source. — Режим доступа: <https://react.dev/learn> (дата обращения: 07.11.2025).
15. Redux Toolkit. Documentation [Электронный ресурс] / Redux Team. — Режим доступа: <https://redux-toolkit.js.org/> (дата обращения: 07.11.2025).
16. TypeScript. Handbook [Электронный ресурс] / Microsoft. — Режим доступа: <https://www.typescriptlang.org/docs/handbook/intro.html> (дата обращения: 22.10.2025).
17. Feature-Sliced Design. Architectural Methodology for Frontend Projects [Электронный ресурс]. — Режим доступа: <https://feature-sliced.design/docs/get-started/overview> (дата обращения: 20.10.2025).
18. MDN Web Docs. WebRTC API [Электронный ресурс] / Mozilla Foundation. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API (дата обращения: 28.10.2025).
19. Vite. Guide [Электронный ресурс] / Evan You. — Режим доступа: <https://vitejs.dev/guide/> (дата обращения: 08.11.2025).
20. Rosenberg, J. SIP: Session Initiation Protocol : RFC 3261 [Электронный ресурс] / J. Rosenberg, H. Schulzrinne, G. Camarillo [и др.]. — IETF, 2002. — Режим доступа: <https://datatracker.ietf.org/doc/html/rfc3261> (дата обращения: 08.11.2025).

ПРИЛОЖЕНИЕ А

Листинг А.1 — Структура sessionSlice

```
export type SessionState = {
  sessionId: string | null;
  userName: string | null;
  userId: string | null;
  userCommonName: string | null;
  userNumbers: string[];
  availStatus: string | null;
  vertoUrl: string | null;
  user: SessionUser | null;
  wsStatus: WsConnectionStatus;
  useVerto: boolean;
};

export const sessionSlice = createSlice({
  name: 'session',
  initialState,
  reducers: {
    setSession(_state, action: PayloadAction<{
      sessionId: string;
      userName: string;
      userId?: string;
      availStatus?: string;
      vertoUrl?: string | null;
      useVerto?: boolean;
    }>) {
      const p = action.payload;
      return {
        sessionId:      p.sessionID,
```

```

        userName:      p.userName,
        userId:        p.userId ?? null,
        userCommonName: null,
        userNumbers:    [],
        availStatus:    p.availStatus ?? null,
        vertoUrl:       p.vertoUrl ?? null,
        user:           null,
        wsStatus:       'disconnected' as const,
        useVerto:       p.useVerto ?? true,
    };
},
setAvailStatus(state, action: PayloadAction<string>) {
    state.availStatus = action.payload;
},
setWsStatus(state, action:
PayloadAction<WsConnectionStatus>) {
    state.wsStatus = action.payload;
},
clearSession() {
    return initialState;
},
},
});
export const { setSession, clearSession,
                setAvailStatus, setWsStatus } =
sessionSlice.actions;
export const selectIsAuthenticated =
(s: RootState) => Boolean(s.session.sessionID);
export const selectSessionID =
(s: RootState) => s.session.sessionID;
export const selectAvailStatus =
(s: RootState) => s.session.availStatus;

```

Листинг A.2 — Компонент AgentRowItem

```
function AgentRowItem({ row, now, showDuration, onQuickDial }: {
  row: AgentRow;
  now: number;
  showDuration: boolean;
  onQuickDial: (info: { number: string; name: string }) => void;
}) {
  const dotClass = presenceToDotClass(row.presence);

  const handleUsernameClick = () => {
    const num = row.numbers[0];
    if (num) onQuickDial({ number: num, name: row.displayName
  });
  };

  return (
    <div className={`user-list__agent ${
      row.presence === 'ONLINE_BUSY'
        ? 'user-list__agent--selected' : ''
    }`} >
      <span className={`status-dot ${dotClass}`} />
      <span
        className="user-list__agent-name--clickable"
        onClick={handleUsernameClick}
      >
        {row.displayName}
      </span>
      <span className="user-list__col--numbers">
        {row.numbers.map((n, i) => (
          <span key={i} className="badge badge--clickable"
```

```

        onClick={() => onQuickDial({
            number: n, name: row.displayName
        })}>
        📞 {n}
    </span>
    )})
</span>
<span className="badge">{row.availStatusLabel}</span>
</div>
);
}

```

Листинг A.3 — Обработчик принятия входящего вызова

```

export async function answerInboundCall(
    callID: string,
    remoteSdp: string,
    dispatch: (a: unknown) => unknown,
) {
    try {
        const { answerSdp } = await createInboundSession(
            callID, remoteSdp
        );
        await vertoClient.answer(callID, answerSdp);
        dispatch(vertoActions.updateCallState({
            callID, state: 'active'
        }));
        stopRingtone();
    } catch (err) {
        console.error('[Verto] Failed to answer incoming call:',
err);
        destroySession(callID);
        dispatch(vertoActions.removeCall(callID));
    }
}

```

```

        stopRingtone();
    }
}

```

Листинг A.4 — Метод инициализации VertoClient

```

export class VertoClient {
    private ws: WebSocket | null = null;
    private rpcId = Math.floor(Math.random() * 1_000_000);
    private pending = new Map<number, PendingRequest>();
    private sessid: string = '';
    private _connected = false;
    private handlers: VertoEventHandler = {};

    get connected() { return this._connected; }

    setHandlers(h: VertoEventHandler) {
        this.handlers = h;
    }

    connect(url: string, login: string,
            password: string): Promise<void> {
        return new Promise((resolve, reject) => {
            this.ws = new WebSocket(url);
            this.ws.onopen = async () => {
                try {
                    await this.doLogin(login, password);
                    this._connected = true;
                    this.handlers.onReady?.();
                    resolve();
                } catch (err) {
                    reject(err);
                    this.disconnect();
                }
            }
        });
    }
}

```



```

        }

    };

    this.ws.onmessage = (e) => this.handleMessage(e.data);
    this.ws.onclose = () => {
        this._connected = false;
        this.cleanupPending();
        this.handlers.onDisconnect?.();
    };
});
}

disconnect() {
    this._connected = false;
    this.ws?.close();
    this.ws = null;
    this.cleanupPending();
}

}

export const vertoClient = new VertoClient();

```

Листинг A.5 — Метод установления исходящего вызова

```

async invite(
    callID: string,
    destinationNumber: string,
    callerIdName: string,
    callerIdNumber: string,
    localSdp: string,
): Promise<{ callID: string }> {
    const dialogParams = {
        callID,

```

```

    destination_number: destinationNumber,
    caller_id_name:      callerIdName,
    caller_id_number:    callerIdNumber,
    incomingBandwidth:   'default',
    outgoingBandwidth:   'default',
    useStereo:           false,
    useVideo:            false,
    screenShare:         false,
    dedEnc:              false,
    audioParams:         {},
    loginParams:         {},
  };

  const result = await this.send('verto.invite', {
    sdp: localSdp,
    dialogParams,
    sessid: this.sessid,
  });

  return { callID: (result.callID as string) ?? callID };
}

```

Листинг А.6 — Конфигурация Redux Store

```

export const store = configureStore({
  reducer: {
    session:      sessionReducer,
    user:         userReducer,
    callGroup:    callGroupReducer,
    bundle:       bundleReducer,
    cdr:          cdrReducer,
    verto:        vertoReducer,
    groupOrder:   groupOrderReducer,
    viewSettings: viewSettingsReducer,
    [api.reducerPath]: api.reducer,
  },
});

```

```

    },
    middleware: (getDefaultMiddleware) =>
        getDefaultMiddleware({
            serializableCheck: false,
        })
        .concat(api.middleware)
        .concat(vertoMiddleware),
    });

export type RootState    = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

```

Листинг А.7 — Утилиты управления настройками

```

interface ServerSettings {
    ringtone?: string;
    endCallSound?: boolean;
}

const LS_STUN = 'settings.useStun';

function loadStun(): boolean {
    try {
        const v = localStorage.getItem(LS_STUN);
        if (v === null) return false;
        return v === 'true';
    } catch { return false; }
}

function parseServerSettings(
    raw: string | undefined | null
): ServerSettings {
    if (!raw) return {};
    try { return JSON.parse(raw) as ServerSettings; }

```

```
    catch { return {};} }  
}  
  
const handleUseStunChange = (v: boolean) => {  
    setUseStun(v);  
    try {  
        localStorage.setItem(LS_STUN, String(v));  
    } catch {}  
};
```